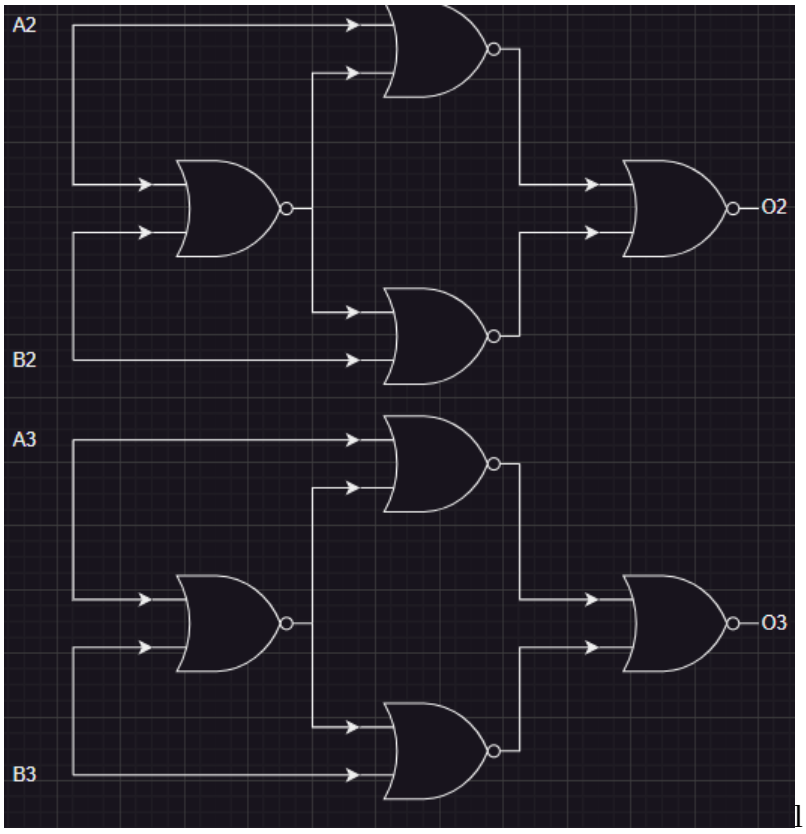
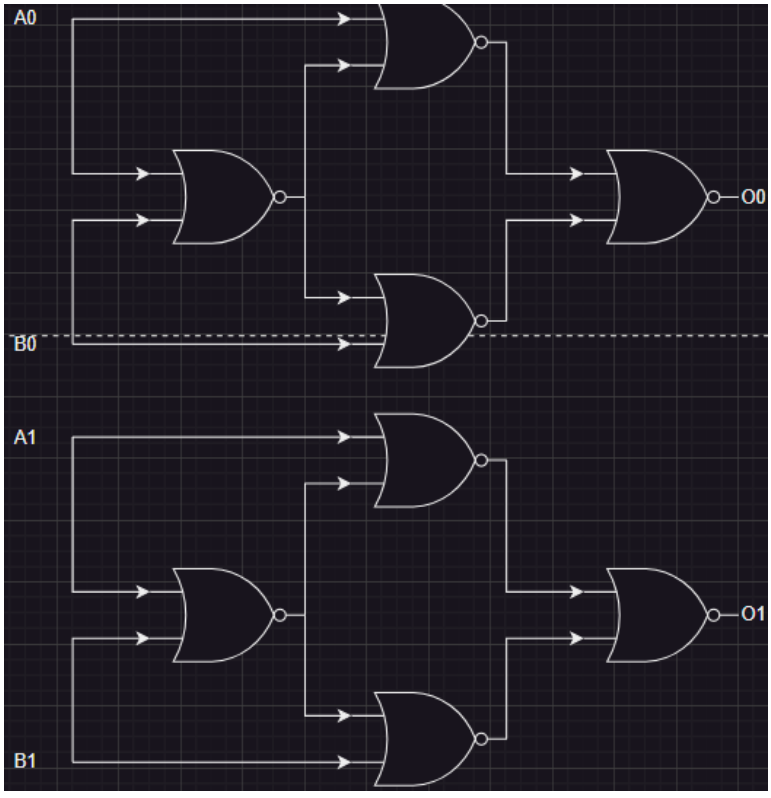
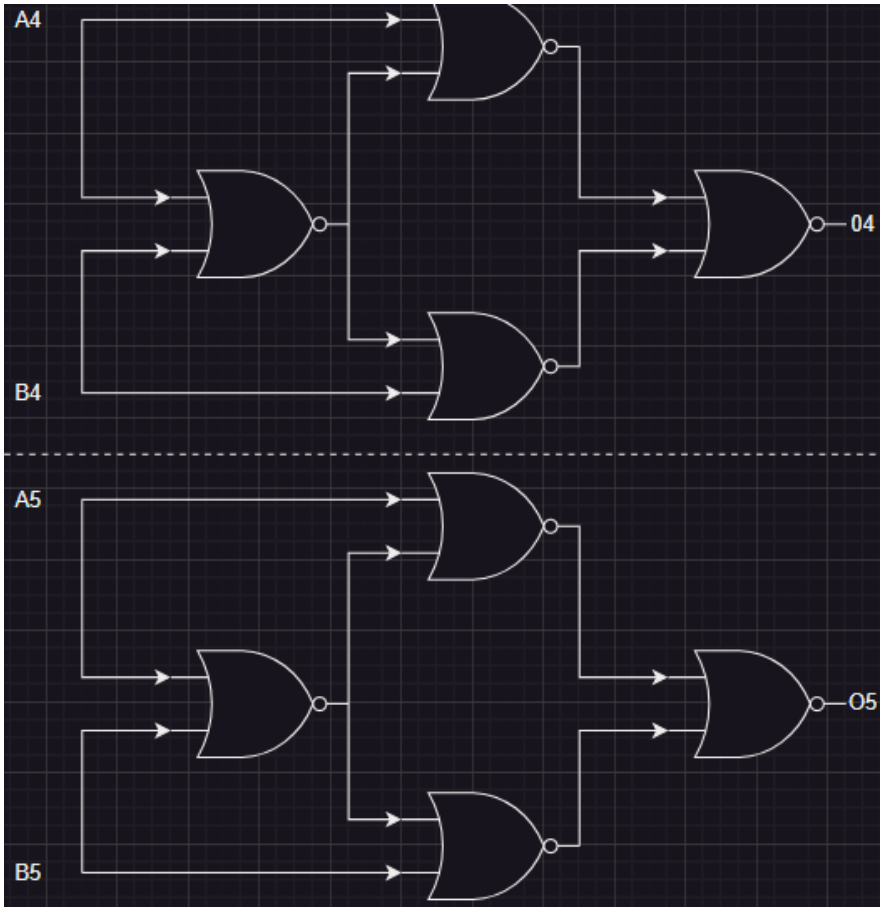
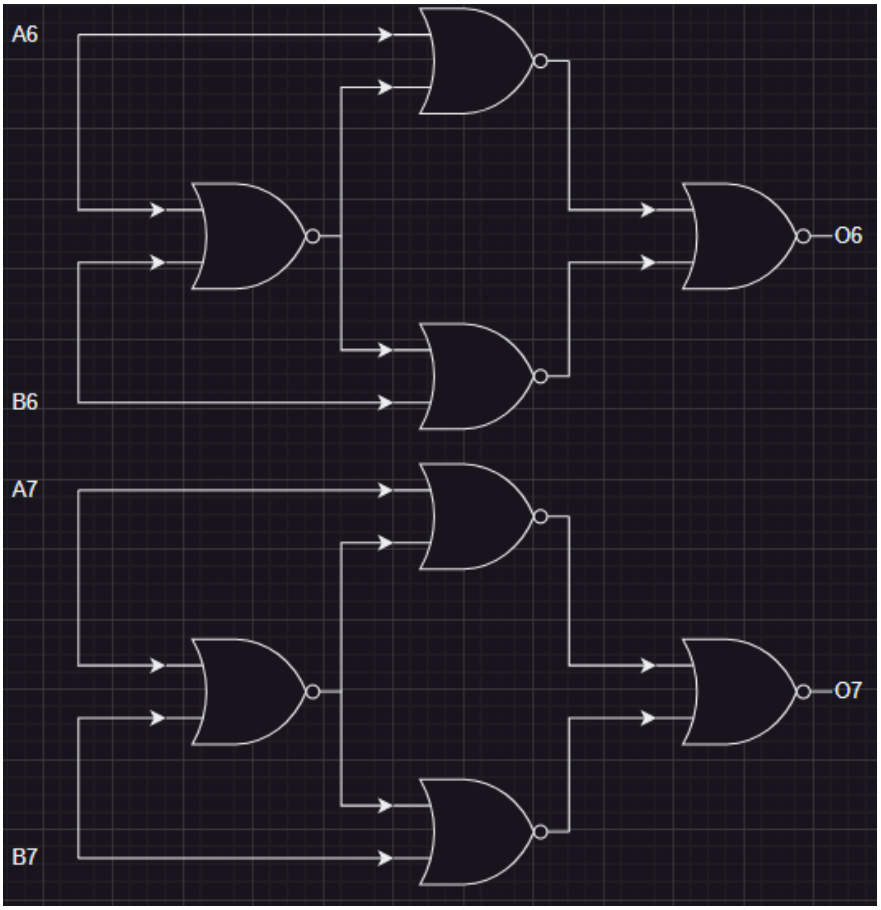
Results

XNOR From NOR only

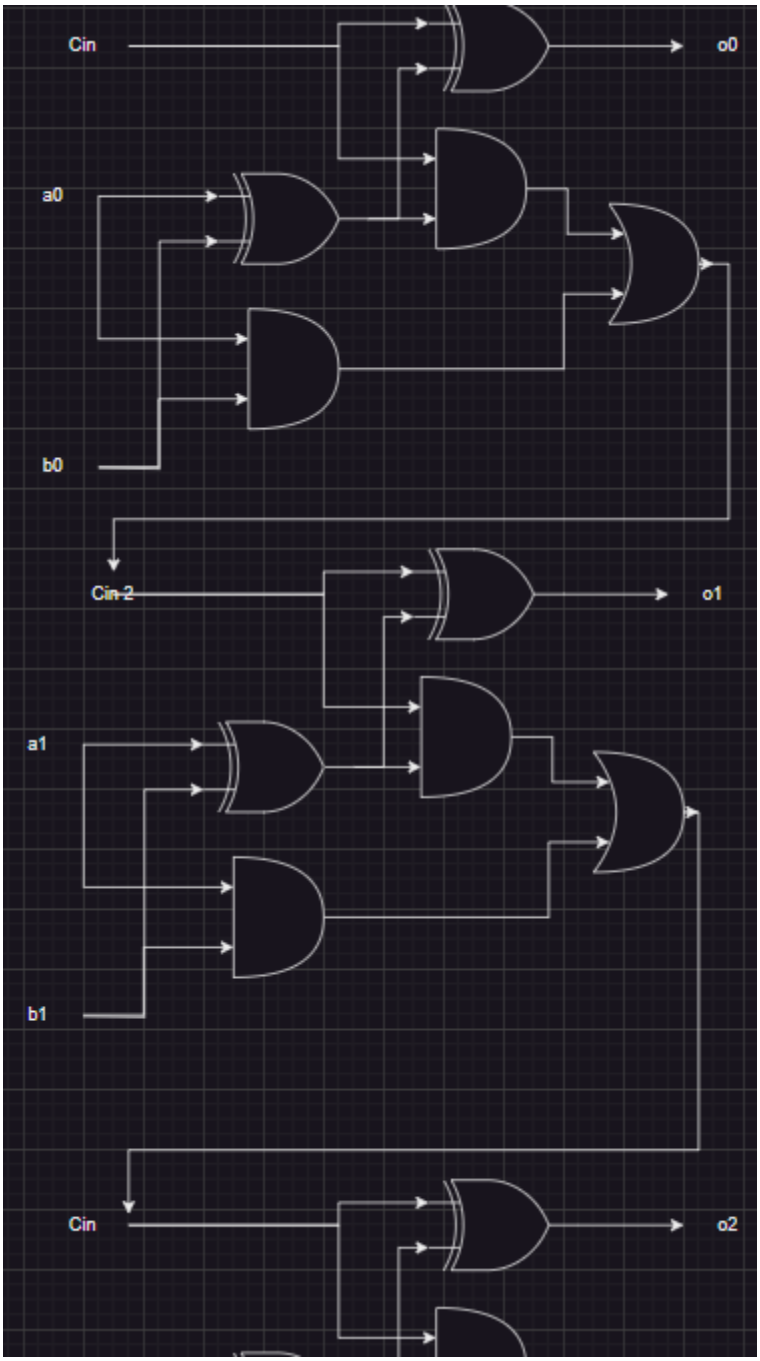


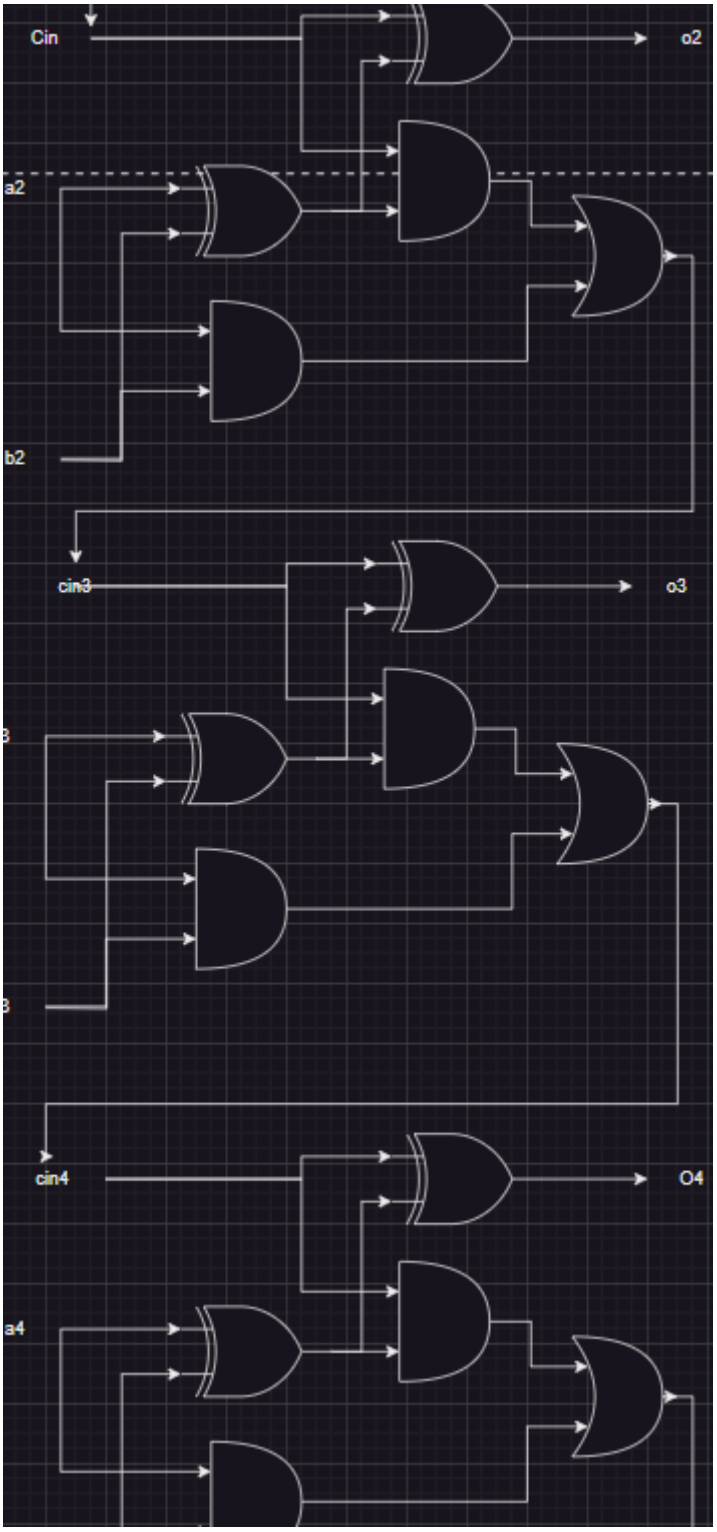


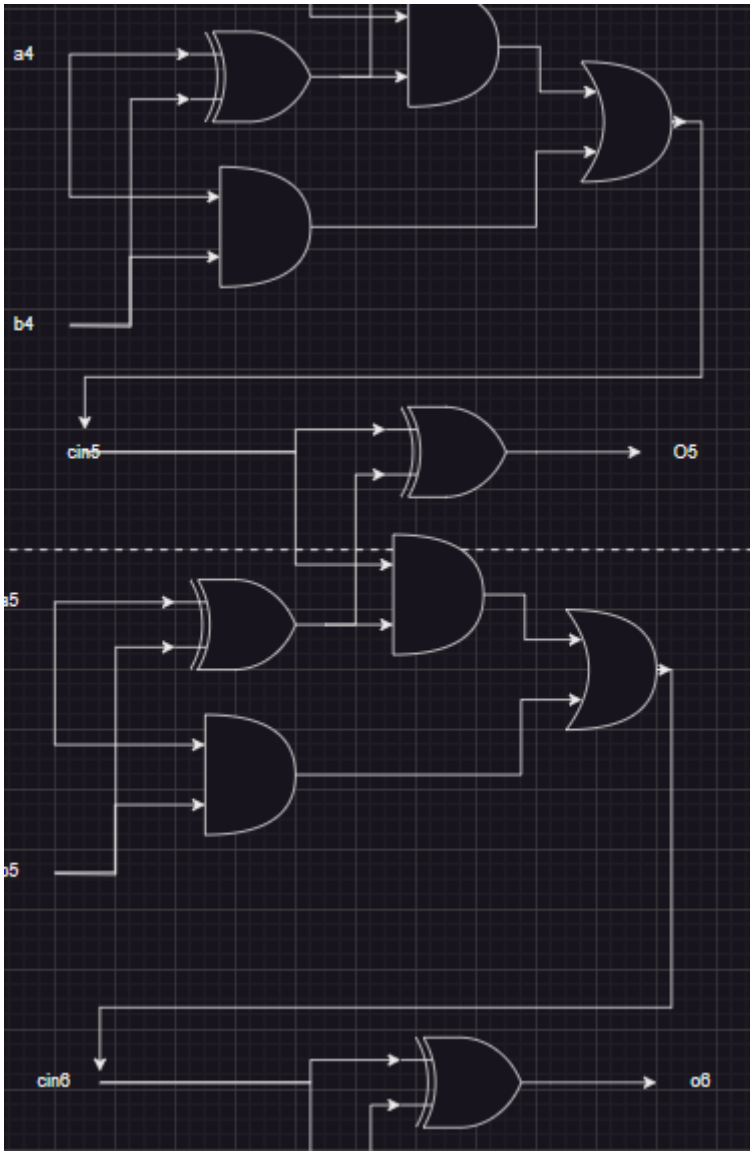

A4
B4
04
A5
B5
O5

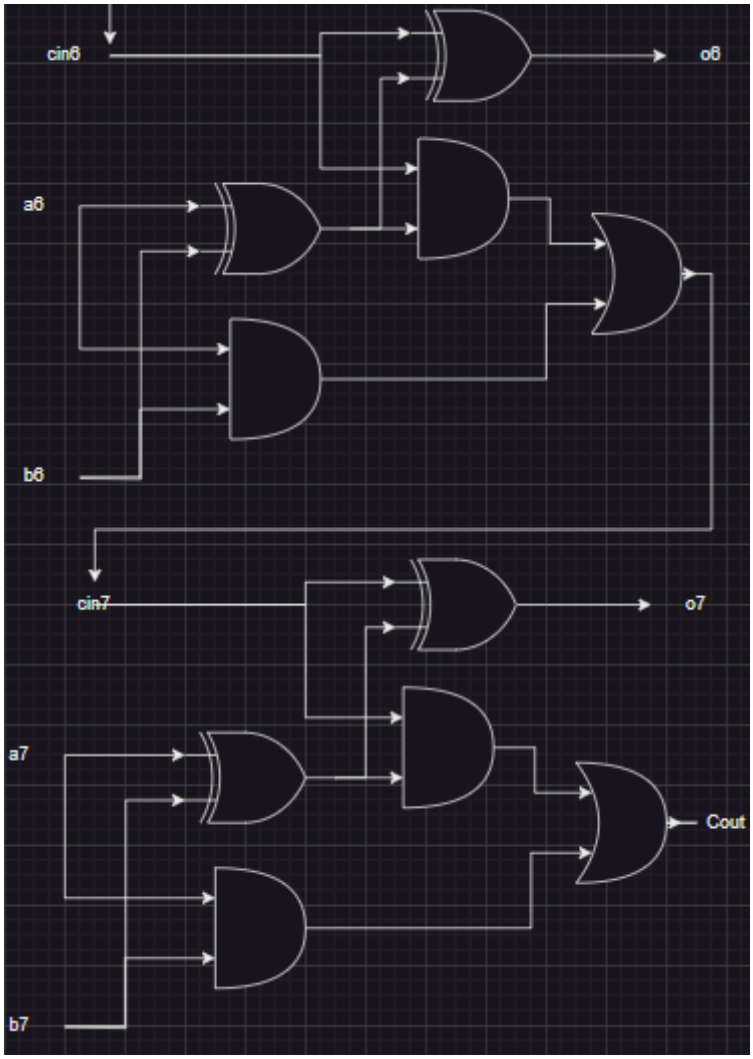

A6
B6
O6
A7
B7
O7

8-Bit Ripple Adder

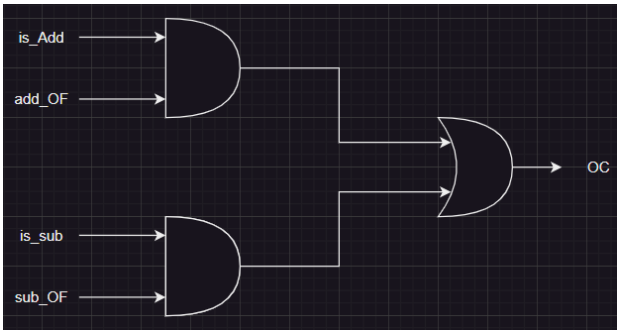



Cin
o2
a2
b2
cin3
o3
cin4
O4
a4

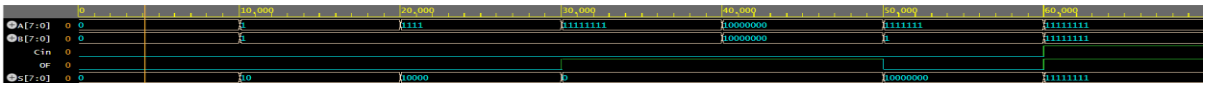

a4
b4
cin5
O5
cin6
o6



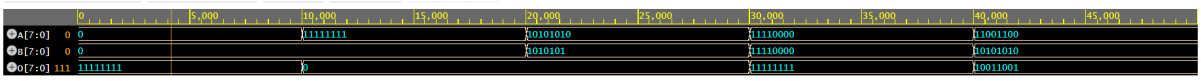
Overflow Checker



Result Adder



Result XNOR



Methodology.

8-bit Ripple Adder

8-bit Ripple Adder have the functionality of adding two 8-bits input and provide an 8-bit output with a special function of indicating whether there is an carryout. The carryout occurs when the summation of both inputs exceed 8-bit, which converts up to 256 in decimal. This is done by extracting the data from the Cout bit

The 8-bit Ripple Adder is done by combining 8 individual one-bit full adder. The construction of 1-bit full adder is done by constructing the truth table as shown as below.

Inputs			Outputs	
A	B	C – IN	Sum	C - Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

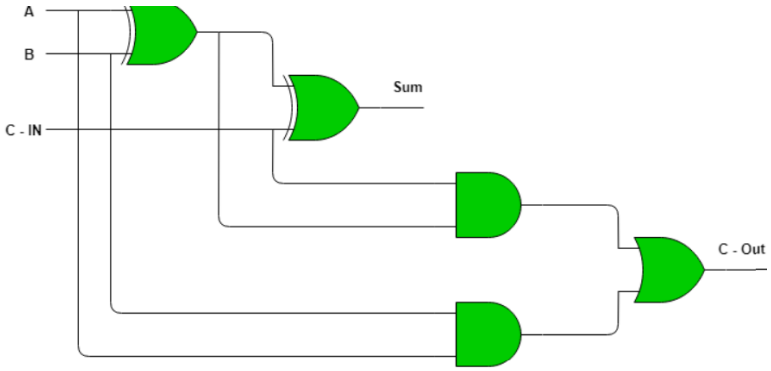
By annalysing the relationship between the input A,B,Cin and output Sum, the following equation can be obtained

$$A'B'Cin + A'BCin' + AB'Cin' + ABCin = Cin(A'B' + AB) + Cin'(A'B + AB')$$
$$= Cin\ XOR(AXORB)$$

At the same time the output Cout can be represented as such

$$AB + ACin + BCin(A + A') = ABCin + AB + ACin + A'BCin$$
$$= AB(1 + Cin) + ACin + A'BCin = AB + ACin + A'BCin$$
$$= AB + ACin(B + B') + A'BCin = ABCin + AB + AB'Cin + A'BCin$$
$$= AB(Cin + 1) + AB'Cin + A'BCin = AB + AB'Cin + A'BCin$$
$$= AB + Cin(AXORB)$$

By combining both the inputs, the following can be obtained:



Full Adder logic circuit.

Implementing the circuit schematics into the Verilog Code file the following can be programmed

```
module OneAdder(sum, c_out, a, b, c_in);
 output sum, c_out;
 input a, b, c_in;
 wire s1, c1, c2;

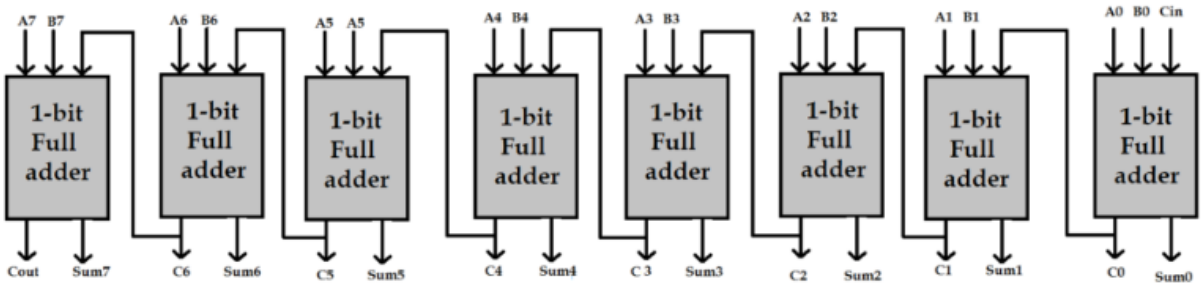
 xor (s1, a, b);
 and (c1, a, b);

 xor (sum, s1, c_in);
 and (c2, s1, c_in);

 or (c_out, c2, c1);
endmodule
```

When the gates are connected in such a way that follows the schematic deisgn of the circuit. The first xor gate and the first and gate serves the purpose of pre proecessing the input that is placed directly into the ALU function module, they analyse the relationship between the input only and without the interference of the Carry In which is being applied at the second half of the module at the XOR and AND gate. As such the Carry In will be taken into consideration after the addition of both results.

By leaving the Cin of 1st bit input and connecting the Cout of 1st bit input to Cin of 2nd bit input, and the Cout of 2nd bit input to Cin of 3rd bit input, a completed 8-bit full adder can be represented as such:



The representation of the above mentioned ALU module can be represented as such

```
OneAdder Adder1(S[0], c[0], A[0], B[0], Cin);
OneAdder Adder2(S[1], c[1], A[1], B[1], c[0]);
OneAdder Adder3(S[2], c[2], A[2], B[2], c[1]);
OneAdder Adder4(S[3], c[3], A[3], B[3], c[2]);
OneAdder Adder5(S[4], c[4], A[4], B[4], c[3]);
OneAdder Adder6(S[5], c[5], A[5], B[5], c[4]);
OneAdder Adder7(S[6], c[6], A[6], B[6], c[5]);
OneAdder Adder8(S[7], Cout, A[7], B[7], c[6]);
```

The logic behind such coding is that we will be using the original self created low level module and represent it as a high level module by reusing the module multiple times. In this case the low level module would be the module named OneAdder and the high level module would be the compilation of these module as shown in the coding above.

XNOR

The XNOR gate has the following truth table:

A	B	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

By observing the output we can obtain the following equation

$$XNOR = XY + X'Y'$$

$$NOT = X\ NOR\ X$$

$$NOR = (X + Y)'$$

$$(X'\ NOR\ Y') = X \cdot Y$$

$$(\text{X NOR Y}) = X + Y$$

Given that A • B= (A'NOR B') and A+B= (A NOR B)' and by expanding AB+AB' we can obtain, by assuming AB to be X and A'B' to be Y

$$AB + A'B' = ((AB)\ NOR\ (A'B'))'$$

By assuming A as X and B as Y the following can be obtained

$$NOT\ A = A\ NOR\ A$$

$$NOT\ B = B\ NOR\ B$$

Hence A • B can be represented as

$$(A\ NOR\ A\ NOR\ B\ NOR\ B)$$

And A'B' can be represented as

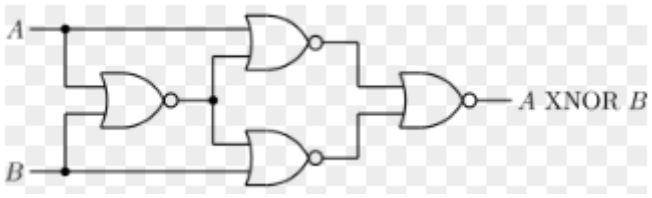
$$(A\ NOR\ B)$$

By combining all equation, we can get

$$A\ XNOR\ B =$$

$$\{[(A\ NOR\ A)\ NOR\ (\ B\ NOR\ B)]NOR\ (A\ NOR\ B)\}\ NOR\ \{[(A\ NOR\ A)NOR\ (\ B\ NOR\ B)]NOR\ (A\ NOR\ B)\}$$

The circuit implementation would be as such:



The above diagram is a one-bit XNOR gate using NOR gates, when the inputs of A and B are connected to different NOR gates to provide the output

```
module DIYXNOR(output out, input input1, input2);
 wire w1, w2, w3;

 nor (w1, input1, input2);
 nor (w2, input1, w1);
 nor (w3, input2, w1);
 nor (out, w2, w3);
endmodule
```

The above code is the low level gate modelling of the schematic representation of XNOR gate from NOR gate. The only gates that are used in the code are NOR gates only and as such coincide with the requirement stated in the assignment guidelines.

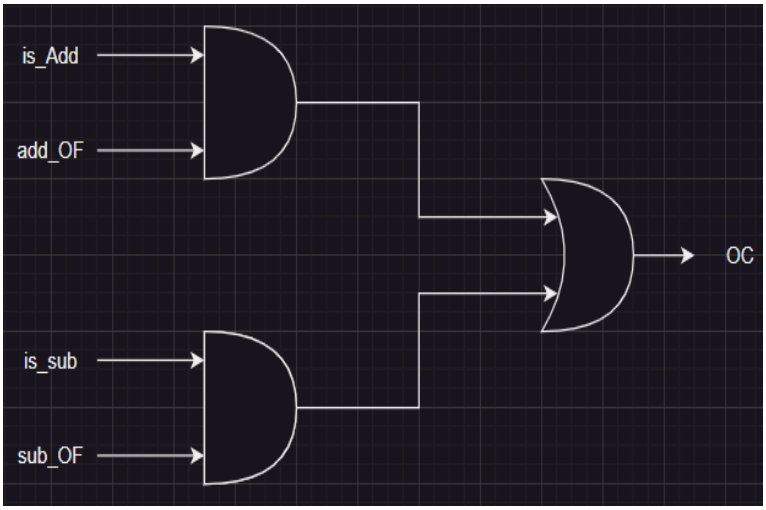
by duplicating the system by 8 times, a 8-bit XNOR gate is created. It serves the function of directly compare the inputs, A and B and generate an output of O, this process is performed bitwise, and each bit does not interfere with one another, the code is shown below as such.

```
module Bit8XNOR(input [7:0] A, B, output [7:0] O);
 DIYXNOR X0(O[0], A[0], B[0]);
 DIYXNOR X1(O[1], A[1], B[1]);
 DIYXNOR X2(O[2], A[2], B[2]);
 DIYXNOR X3(O[3], A[3], B[3]);
 DIYXNOR X4(O[4], A[4], B[4]);
 DIYXNOR X5(O[5], A[5], B[5]);
 DIYXNOR X6(O[6], A[6], B[6]);
 DIYXNOR X7(O[7], A[7], B[7]);
endmodule
```

The implementation of low level modelling into a higher level modelling is done by utilizing the low level gate level module named DIYXNOR over several times and resulting in as such.

OF Flag

An additional function, the OF Flag is used in our assignment. OF Stands for Overflow Flag which will be indicated when an Overflow or Carry Out is being observed. The schematic of the design is applied as such



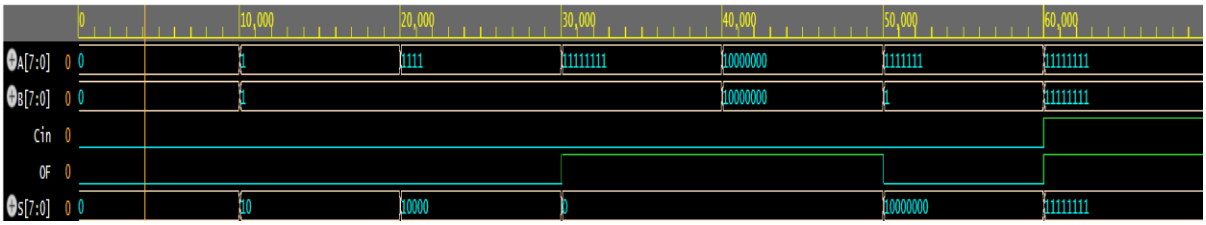
is_add ensures that the ALU is performing the Addition Function module rather than anything else when is_sub ensures that the Subtraction function is being carried out. Such is to prevent accidental trigger when the OF flag is raised when other modules are preforming their functions. The OR gate is added in order to ensure that the OC flag(Overflow or Carry) flag is raised when either Overflow or Carryout is present in the system.

```
// Flag selector logic (just like output mux
wire f_add_c, f_sub_c;
and (f_add_c, is_add, add_OF);
and (f_sub_c, is_sub, sub_OF);
or (OC, f_add_c, f_sub_c);
```

The coding shown above displays how the gates are interconnected, where the output of both and gates f_add_c and f_sub_c is connected as input for the OR gate and the output is once again being displayed out as OC to serve the purpose of displaying to the user that a Overflow or Carry is present in the circuit.

Discussion

A test had been implemented solely on the 8-bit ripple adder; the result is shown below as



Where the A, B and Cin is the input and S is output when OF symbolizes the overflow flag. By observing the results, it is certified that the 8-bit full adder is working. For example, when the input is 0 and 0, the output presents a total of 0, and when the input is 1 and 1, the output that is being recorded is 1+1=10 in terms of binary. Lastly when the input is 10000000+10000000, the resulted output is 0 with a symbol of carry out flag OF being raised as 1. This symbolizes that the full adder will work under all sorts of conditions.

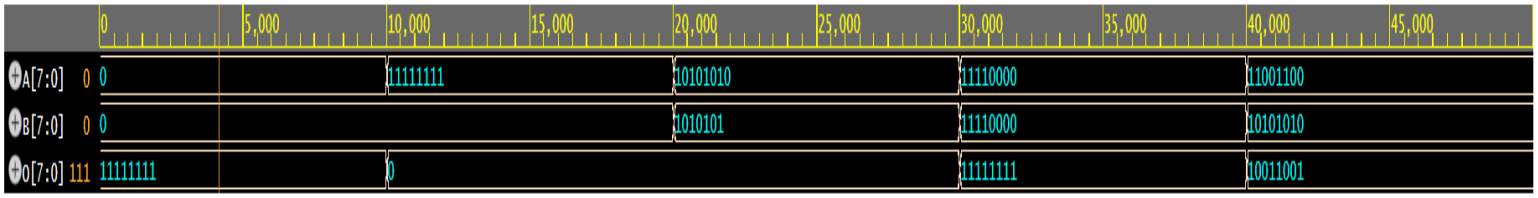
The reason behind such result is that an addition of 1 and 1 will result in a 10, however as the maximum output of the module can only takes up to 8-bits and hence the additional bit in the output when the addition of 1 and 1 occur in the most significant bit, an additional extra bit (the ninth bit) is being created, hence causing the overflow flag OF to be raised

We had assumed that the overflow and Carry out to be used in the same indicator known as the Overflow flag going by the name OF. For the 8th bit output, it is used to indicate carry out. The purpose of such is to symbolizes that there is an overflow in digit and should be flagged out as an anomaly for further display purposes. In this ALU, we had designed a flag specifically for this purpose alongst with the overflow error for 2's compliment subtracton. Which we will be explaining further in the compilation part of the report.

Several precautions have to be taken before utilizing this particular module. As this module does not uses 2's complement during the calculation of results, it will not be able to support any inputs that display a value of negativity. Furthermore the limitation that the module has is that is can only support up to 8 bits of input and will experience an overflow which might cause the module to malfunction if a more than 8-bit input is being created into the system. However, as we does not wish to have experience an output overflow despite the input of being merely 8-bit during two 8-bit overflow addition, an OF flag is being utilized and created.

XOR

The result of a testbench is attached below to display the reliability of our design:



In the graph display, as 00000000 is indicated as 0 and for the input at first second: when the input of all bits is as 0 for A and B, the output is 1 as XNOR indicates that when A is 0 and B is 0, the output is 1. At the second second, when all inputs of A is 1 and all inputs of 0 is 0, the output is 0 through out as 0 and 1 results in 0 in XNOR gate. Lastly at 5th second, when the input is 11001100 for A and 10101010 for B, a clear bitwise comparison is shown as at the first bit starting from the most significant bit 1 and 1 compare the result is 1, and the 2nd bit comparison is 1 and 0, resulting in 0, for the 3rd bit is 0 and 1 resulting in 0, when for the 4th bit is 0 and 0 resulting in 1, and a 1 and 1 comparison for 5th bit results in a 1 when the result of 6th and 7th bit is 0 due to a 1 and 0 comparison and a 0 and 1 comparison respectively. For the least significant bit the comparison between two 0s resulted in a 1, all these shows that our design of the XNOR gate is working and of optimum condition.

The reason behind such is that the input is being processed and compared bit by bi, which means that the module will only cover one bit at a time and bits does not affect or interfere with one another. Hence as such no carry out or overflow flag would be raised between different bits, in another words, different bits are not interconnected. As such our module serves the capability of a XOR comparison between two different inputs which can be utilized in many places such as adder and subtractor.

Several precautions should be taken before utilizing the module for multiple purposes. For example, as the module is a 8-bit input model, any input that exits 8-bit will be rejected. Unlike the adder and subtractor that we has to limit the output to be only at 8-bit and raise an overflow if anything unwanted happens, as this particular module is an XOR gate combination, no additional output will be produced as a result and hence the overflow flag is not being implemented or created for this assignment.